

MODEL MATEMATIKA

Artificial Neural Network (ANN)

Prediksi Harga Saham

Mata Kuliah: Machine Learning

Dataset: NIFTY 50 Index (NSE India)

Framework: TensorFlow / Keras (Python)

1. Pendahuluan

Prediksi harga saham merupakan permasalahan time-series yang bersifat non-linear dan sangat volatil. Pada implementasi ini, digunakan model Feedforward Artificial Neural Network (ANN) berbasis sliding window untuk mempelajari pola historis harga penutupan (Close Price) dan melakukan prediksi satu hari ke depan.

Alur pemodelan secara keseluruhan terdiri dari lima tahap utama:

- Normalisasi data (MinMaxScaler)
- Pembentukan pasangan input-output (Sliding Window)
- Propagasi maju (Forward Pass) melalui jaringan ANN
- Minimisasi Loss menggunakan optimizer Adam
- Evaluasi performa model (RMSE, MAE, MAPE, DA)

2. Normalisasi Data (MinMaxScaler)

Sebelum dimasukkan ke jaringan, data harga saham dinormalisasi ke rentang [0, 1] menggunakan metode Min-Max Normalization. Hal ini penting untuk mempercepat konvergensi gradient descent dan mencegah dominasi fitur dengan skala besar.

2.1 Formula Normalisasi

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Keterangan: x = nilai Close Price asli, x_{min} = nilai minimum dalam dataset, x_{max} = nilai maksimum dalam dataset, x' = nilai ternormalisasi dalam rentang [0, 1]

2.2 Formula Invers Normalisasi

Setelah prediksi dihasilkan dalam skala [0,1], nilai dikembalikan ke skala harga asli menggunakan:

$$x = x' \times (x_{\max} - x_{\min}) + x_{\min}$$

Keterangan: Digunakan pada output prediksi sebelum menghitung metrik evaluasi (RMSE, MAE, MAPE).

2.3 Implementasi pada Kode

Pada kode Python menggunakan sklearn:

```
scaler = MinMaxScaler(feature_range=(0,1))
data_scaled = scaler.fit_transform(data)
```

3. Pembentukan Input: Sliding Window

Model ANN membutuhkan input berupa vektor, bukan satu titik data. Metode sliding window digunakan untuk mengubah time-series menjadi pasangan (X, y) yang dapat dipelajari secara supervised.

3.1 Definisi Window

Diberikan data ternormalisasi $C' = \{C'_1, C'_2, \dots, C'_n\}$, dengan ukuran window $w = 5$, maka:

$$X^{(i)} = [C'_i, C'_{i+1}, C'_{i+2}, C'_{i+3}, C'_{i+4}]$$

$$Y^{(i)} = C'_{i+5}$$

Keterangan: $X^{(i)}$ adalah vektor input (5 harga penutupan berurutan), $Y^{(i)}$ adalah target prediksi (harga penutupan hari ke-(i+5)), $i = 1, 2, \dots, n-w$

3.2 Dimensi Data

Variabel	Dimensi	Keterangan
X (input)	$(N - w) \times w = (N-5) \times 5$	Matriks input semua sampel
y (target)	$(N - w) \times 1 = (N-5) \times 1$	Vektor target semua sampel
X_train	$80\% \times (N-5) \times 5$	Data latih (80%)
X_test	$20\% \times (N-5) \times 5$	Data uji (20%)

4. Arsitektur Jaringan ANN

Model yang digunakan adalah Multilayer Perceptron (MLP) dengan 2 hidden layer. Jaringan ini merupakan fungsi komposit yang memetakan vektor input x berdimensi 5 ke output skalar $\hat{y}$.

4.1 Ringkasan Arsitektur

Layer	Tipe	Neuron	Aktivasi	Parameter
Input	Input Layer	5	-	0
Hidden 1	Dense	10	ReLU	$W[1]: (10 \times 5)$, $b[1]: (10 \times 1) \Rightarrow 60$ param
Hidden 2	Dense	5	ReLU	$W[2]: (5 \times 10)$, $b[2]: (5 \times 1) \Rightarrow 55$ param
Output	Dense	1	Linear	$W[3]: (1 \times 5)$, $b[3]: (1 \times 1) \Rightarrow 6$ param

Total parameter yang dapat dilatih: $60 + 55 + 6 = 121$ parameter.

4.2 Forward Pass - Layer 1 (Hidden, 10 Neuron, ReLU)

Setiap neuron pada layer pertama menghitung kombinasi linear dari input, kemudian ditransformasi dengan fungsi ReLU:

$$Z^{[1]} = W^{[1]}X + b^{[1]}$$

$$a^{[1]} = \text{RELU}(Z^{[1]}) = \max(0, Z^{[1]})$$

Keterangan: $W^{[1]}$ berukuran (10×5) , $b^{[1]}$ berukuran (10×1) , $Z^{[1]}$ dan $a^{[1]}$ berukuran (10×1) . Operasi dilakukan element-wise untuk fungsi ReLU.

4.3 Forward Pass - Layer 2 (Hidden, 5 Neuron, ReLU)

$$Z^{[2]} = W^{[2]}X + b^{[2]}$$

$$a^{[2]} = \text{RELU}(Z^{[2]})$$

Keterangan: $W^{[2]}$ berukuran (5×10) , $b^{[2]}$ berukuran (5×1) , $a^{[2]}$ berukuran (5×1) .

4.4 Forward Pass - Layer Output (1 Neuron, Linear)

Layer output tidak menggunakan fungsi aktivasi (linear/identity), sesuai untuk regresi:

$$\hat{Y} = W^{[3]}a^{[2]} + b^{[3]}$$

Keterangan: $W^{[3]}$ berukuran (1×5) , $b^{[3]}$ adalah skalar, $\hat{Y}$ adalah prediksi harga ternormalisasi.

4.5 Persamaan Lengkap (End-to-End)

Secara keseluruhan, model ANN dapat ditulis sebagai fungsi komposit:

$$\hat{Y} = W^{[3]} \text{ReLU}(W^{[2]} \text{ReLU}(W^{[1]}X + b^{[1]}) + b^{[2]}) + b^{[3]}$$

5. Fungsi Aktivasi ReLU

Fungsi Rectified Linear Unit (ReLU) dipilih karena mengatasi vanishing gradient problem yang umum terjadi pada sigmoid/tanh, dan memiliki komputasi yang sangat efisien.

5.1 Definisi Matematis

$$\text{ReLU}(z) = \max(0, z) = \begin{cases} z, & \text{jika } z > 0 \\ 0, & \text{jika } z \leq 0 \end{cases}$$

5.2 Turunan ReLU (untuk Backpropagation)

$$d(\text{ReLU})/dz = \begin{cases} 1, & \text{jika } z > 0 \\ 0, & \text{jika } z \leq 0 \end{cases}$$

Keterangan: Turunan ReLU bernilai 1 atau 0, sehingga tidak menyebabkan gradient vanishing seperti sigmoid.

6. Fungsi Loss: Mean Squared Error (MSE)

Fungsi loss yang digunakan adalah Mean Squared Error (MSE), dioptimalkan selama proses training. MSE mengukur rata-rata kuadrat selisih antara nilai prediksi dan nilai aktual pada data training.

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)})^2$$

Keterangan: N = jumlah sampel training, $\hat{y}^{(i)}$ = prediksi model untuk sampel ke- i (skala ternormalisasi), $y^{(i)}$ = nilai target aktual (skala ternormalisasi).

Dalam kode Keras, loss ini ditetapkan dengan:

```
model.compile(optimizer='adam', loss='mse')
```

7. Optimizer: Adam (Adaptive Moment Estimation)

Adam adalah optimizer adaptif yang menggabungkan momentum orde pertama (mean gradient) dan momentum orde kedua (variance gradient). Adam merupakan default optimizer pada Keras dan umumnya memberikan konvergensi lebih cepat daripada SGD biasa.

7.1 Langkah Update Adam

Pada setiap iterasi t , untuk setiap parameter θ :

Langkah 1 - Hitung gradient loss terhadap parameter:

$$g_t = dL/d_{\theta}$$

Langkah 2 - Update moment orde pertama (exponential moving average of gradients):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

Langkah 3 - Update moment orde kedua (exponential moving average of squared gradients):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Langkah 4 - Koreksi bias (bias correction) untuk kompensasi inisialisasi nol:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Langkah 5 - Update parameter:

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

7.2 Nilai Default Hyperparameter Adam (Keras)

Hyperparameter	Simbol	Nilai Default	Keterangan
Learning rate	eta (η)	0.001	Ukuran langkah update parameter
Beta 1	beta_1	0.900	Decay rate moment pertama (momentum)
Beta 2	beta_2	0.999	Decay rate moment kedua (RMSProp)
Epsilon	epsilon	1e-7	Konstanta kecil, mencegah pembagian nol

8. Early Stopping

Untuk mencegah overfitting, digunakan teknik Early Stopping yang menghentikan proses training apabila nilai loss tidak mengalami penurunan dalam sejumlah epoch berturut-turut (patience).

STOP jika: $L(\text{epoch}_t) \geq L(\text{epoch}_{t-1})$ selama 'patience' epoch berturut-turut

Keterangan: Pada kode ini: `EarlyStopping(monitor='loss', patience=10)`. Training dihentikan jika training loss tidak turun selama 10 epoch berturut-turut.

Dalam kode:

```
early_stop = EarlyStopping(monitor='loss', patience=10)
```

9. Metrik Evaluasi Model

Setelah training selesai, performa model dievaluasi pada data testing menggunakan empat metrik. Semua metrik dihitung pada skala harga asli (setelah inverse transform).

9.1 Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2}$$

Keterangan: Satuan sama dengan data asli (harga saham). Sensitif terhadap outlier besar. Semakin kecil nilai RMSE, semakin baik model.

9.2 Mean Absolute Error (MAE)

$$MAE = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|$$

Keterangan: Rata-rata selisih absolut antara prediksi dan aktual. Lebih robust terhadap outlier dibandingkan RMSE.

9.3 Mean Absolute Percentage Error (MAPE)

$$MAPE = \frac{1}{N} \sum_{i=1}^N \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100\%$$

Keterangan: Mengukur error dalam persentase relatif terhadap nilai aktual. Mudah diinterpretasikan (contoh: MAPE 2% berarti rata-rata error 2% dari harga aktual).

9.4 Directional Accuracy (DA)

$$DA = \frac{1}{N-1} \sum_{i=2}^N 1[\text{sign}(\hat{y}_i - \hat{y}_{i-1}) = \text{sign}(y_i - y_{i-1})] \times 100\%$$

Keterangan: Mengukur seberapa sering model memprediksi arah pergerakan harga (naik/turun) dengan benar. Nilai 1 menandakan prediksi arah benar, 0 salah. Metrik tambahan yang relevan untuk trading.

9.5 Ringkasan Interpretasi Metrik

Metrik	Formula Ringkas	Satuan	Interpretasi Ideal
RMSE	$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2}$	Harga (IDR/USD)	Sekecil mungkin
MAE	$MAE = \frac{1}{N} \sum_{i=1}^N \hat{y}_i - y_i $	Harga (IDR/USD)	Sekecil mungkin
MAPE	$MAPE = \frac{1}{N} \sum_{i=1}^N \left \frac{y_i - \hat{y}_i}{y_i} \right \times 100\%$	Persen (%)	< 10% = sangat baik
DA	% arah prediksi benar	Persen (%)	> 50% = lebih baik dari random

10. Prediksi Satu Langkah ke Depan

Setelah model terlatih, prediksi harga hari berikutnya dilakukan menggunakan 5 data terakhir dari dataset sebagai input:

$$X_{next} = [C'_{n-4}, C'_{n-3}, C'_{n-2}, C'_{n-1}, C'_n]$$

$$\hat{y}_{next}^{scaled} = f_{ANN}(X_{next})$$

$$\hat{y}_{next} = \hat{y}_{next}^{scaled} x(x_{max} - x_{min}) + x_{min}$$

Keterangan: f_{ANN} adalah fungsi forward pass model ANN yang telah terlatih. Hasil akhir adalah prediksi harga penutupan hari berikutnya dalam satuan asli.

Implementasi pada kode:

```
last_data = data_scaled[-5:].reshape(1,5)
next_price_scaled = model.predict(last_data)
next_price = scaler.inverse_transform(next_price_scaled)
```